

Reporte de Análisis

Materia: Análisis y diseño de algoritmos

Alumno: Jaziel Ubaldo Ángel Quintero

Profe: Morgan

Carrera: Ingeniería de Software

Marzo 2026

Introducción

Este reporte analiza los ocho algoritmos de ordenamiento presentados en la plataforma VisualAlgo (visualgo.net/en/sorting). Para cada algoritmo se describe su funcionamiento, se incluye el pseudocódigo de la plataforma, se presenta el análisis de su complejidad mediante la notación Big O y se discute su desempeño según el tipo de entrada.

La notación Big O describe el comportamiento de un algoritmo cuando el tamaño de la entrada (n) crece. Las tres cotas utilizadas son: O (cota superior / peor caso), Ω (cota inferior / mejor caso) y Θ (cota ajustada, cuando ambas coinciden). El análisis cubre mejor caso, caso promedio y peor caso para cada algoritmo, así como el espacio adicional de memoria requerido.

Al final del reporte se incluye una tabla comparativa y un análisis general que permite identificar el algoritmo más adecuado según el contexto de uso.

1. Bubble Sort

Descripción

Recorre la lista repetidamente comparando pares de elementos adyacentes. Si un par está desordenado, los intercambia. Cada pasada completa coloca el elemento más grande al final ("burbujea" hacia arriba). Se repite hasta que no se realicen intercambios

Pseudocódigo

```
hacer
    intercambiado = falso
    para i = 1 hasta indexOfLastUnsortedElement-1
        si leftElement > rightElement
            intercambiar(elementoizquierdo, elementoderecho)
            intercambiado = verdadero; ++contador de intercambios
    mientras se intercambiaban
```

Análisis de complejidad — Notación Big O

Mejor caso	Caso promedio	Peor caso	Espacio
$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Desglose:

- Peor caso $O(n^2)$: la lista está en orden inverso. Cada pasada realiza $n-1, n-2, \dots, 1$ comparaciones. Total: $n(n-1)/2$ operaciones.
- Mejor caso $O(n)$: la lista ya está ordenada. La bandera 'intercambiado' permite terminar tras una sola pasada sin intercambios.
- Caso promedio $O(n^2)$: en datos aleatorios, se esperan aproximadamente $n^2/4$ comparaciones e intercambios.
- Espacio $O(1)$: solo usa una variable temporal para los intercambios, sin memoria adicional.

2. Selection Sort

Descripción

Divide la lista en una parte ordenada (al inicio) y una no ordenada. En cada iteración busca el elemento mínimo de la parte no ordenada y lo coloca al final de la parte ordenada mediante un intercambio directo

Pseudocódigo

```
repetir (numOfElements - 1) veces
    establecer el primer elemento no ordenado como el mínimo
    para cada uno de los elementos no ordenados
        si elemento < currentMinimum
            establecer elemento como nuevo mínimo
    Intercambiar mínimo con la primera posición no ordenada
```

Análisis de complejidad — Notación Big O

Mejor caso	Caso promedio	Peor caso	Espacio
$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

Desglose:

- Todos los casos $O(n^2)$: siempre realiza $n(n-1)/2$ comparaciones sin importar el estado inicial de la lista. No existe optimización para el mejor caso.
- Ventaja clave: realiza como máximo $n-1$ intercambios, el mínimo posible. Útil cuando las escrituras en memoria son costosas.
- Espacio $O(1)$: in-place, sin memoria adicional.
- No es estable: el intercambio puede alterar el orden relativo de elementos iguales.

3. Insertion Sort

Descripción

Construye la lista ordenada elemento por elemento. Toma cada nuevo elemento y lo inserta en su posición correcta dentro de la parte ya ordenada, desplazando los elementos mayores hacia la derecha. Similar a ordenar cartas en la mano

Pseudocódigo

```
marcar el primer elemento como ordenado
para cada elemento no ordenado X
    'extraer' el elemento X
    para j = lastSortedIndex hasta 0
        Si el elemento actual j > X
            mover el elemento ordenado a la derecha en 1 posición
        Salir del bucle e insertar X aquí
```

Análisis de complejidad — Notación Big O

Mejor caso	Caso promedio	Peor caso	Espacio
$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Desglose:

- Peor caso $O(n^2)$: lista en orden inverso. El elemento i requiere i desplazamientos: $1+2+\dots+(n-1) = n(n-1)/2$ operaciones.
- Mejor caso $O(n)$: lista ya ordenada. El bucle interno nunca ejecuta, solo $n-1$ comparaciones.
- Confirmación experimental: con $n=10$ se registran 23 movimientos; con $n=200$ se registran 7,575 movimientos — crecimiento de $329x$ para un aumento de $20x$ en n , confirmando $O(n^2)$ ya que $20^2=400 \approx 329$.
- Es más eficiente que Bubble Sort en la práctica. Es la base del algoritmo Timsort (Python, Java)

4. Merge Sort

Descripción

Aplica la estrategia Divide y Vencerás: divide recursivamente la lista en mitades hasta tener sublistas de un elemento (ya ordenadas por definición) y luego las fusiona ordenadamente. La eficiencia viene de la fase de fusión

Pseudocódigo

```
dividir cada elemento en particiones de tamaño 1
fusionar recursivamente particiones adyacentes
para i = leftPartIdx hasta rightPartIdx
    si leftPartHeadValue <= rightPartHeadValue
        copiar izquierdaPartHeadValue
    De lo contrario: copiar rightPartHeadValue; incrementar
Copiar los elementos de vuelta al array original.
```

Análisis de complejidad — Notación Big O

Mejor caso	Caso promedio	Peor caso	Espacio
$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

Desglose:

- Todos los casos $O(n \log n)$: la división siempre parte a la mitad ($\log_2 n$ niveles). En cada nivel se fusionan n elementos en total. Resultado: $n \times \log_2 n$ operaciones garantizadas.
- Es el único algoritmo de comparación que garantiza $O(n \log n)$ incluso en el peor caso.
- Desventaja: requiere $O(n)$ de memoria adicional para el arreglo auxiliar de fusión.
- Es estable: mantiene el orden relativo de elementos iguales durante la fusión.

5. Quick Sort

Descripción

Selecciona un elemento 'pivote' (primer elemento) y particiona el arreglo: los menores al pivote quedan a su izquierda y los mayores a su derecha. Luego aplica el mismo proceso recursivamente a cada partición

Pseudocódigo

```
para cada partición (no ordenada)
    establecer el primer elemento como pivote
    índice_tienda = índice_pivote + 1
    para i = pivotIndex+1 hasta rightmostIndex
        si ((a[i] < a[pivote]) o (igual pero 50% de suerte))
            intercambiar(i, índice_tienda); ++ índice de tienda
    intercambio(pivote, índice_tienda-1)
```

Análisis de complejidad — Notación Big O

Mejor caso	Caso promedio	Peor caso	Espacio
$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Desglose:

- Caso promedio $O(n \log n)$: si el pivote divide aproximadamente a la mitad, hay $\log_2 n$ niveles de recursión con $O(n)$ trabajo por nivel.
- Peor caso $O(n^2)$: ocurre cuando el pivote siempre es el mínimo o máximo (lista ya ordenada con pivote = primer elemento). Genera n niveles con $O(n)$ trabajo cada uno.
- Ventaja práctica: es in-place (solo $O(\log n)$ de pila de recursión) y tiene constantes menores que Merge Sort, siendo más rápido en promedio.
- Solución al peor caso: aleatorizar la elección del pivote (ver Random Quick Sort).

6. Random Quick Sort

Descripción

Variante de Quick Sort que elige el pivote de forma aleatoria en lugar de usar siempre el primer elemento. Esta modificación elimina la posibilidad de construir un input que degrade el algoritmo al peor caso

Pseudocódigo

```
para cada partición (no ordenada)
  Seleccionar aleatoriamente el elemento pivote e índice_pivote
  índice_tienda = índice_pivote + 1
  para i = pivotIndex+1 hasta rightmostIndex
    si ((a[i] < a[pivote]) o (igual pero 50% de suerte))
      intercambiar(i, índice_tienda); ++ índice de tienda
  intercambio(pivote, índice_tienda-1)
```

Análisis de complejidad — Notación Big O

Mejor caso	Caso promedio	Peor caso	Espacio
$O(n \log n)$	$O(n \log n)$	$O(n \log n) *$	$O(\log n)$

Desglose:

- El peor caso $O(n^2)$ sigue existiendo teóricamente, pero su probabilidad es $1/n!$ — prácticamente imposible para cualquier input.
- El análisis probabilístico demuestra que el número esperado de comparaciones es $2n \ln(n) \approx 1.39 \times n \log_2(n)$.
- Al aleatorizar, ningún input específico puede forzar el peor caso: se convierte en $O(n \log n)$ con alta probabilidad.
- Es la variante de Quick Sort recomendada para uso general. Misma implementación que Quick Sort con un swap aleatorio inicial.

7. Counting Sort

Descripción

No compara elementos entre sí. En su lugar, cuenta cuántas veces aparece cada valor distinto usando un arreglo auxiliar de frecuencias. Luego recorre ese arreglo para reconstruir la lista ordenada.

Pseudocódigo

```
crear matriz de claves (conteo)
para cada elemento de la lista
    Incrementa el contador correspondiente en 1.
para cada contador, comenzando por la clave más pequeña
    mientras el contador sea distinto de cero
        restaurar elemento a la lista
        disminuir el contador en 1
```

Análisis de complejidad — Notación Big O

Mejor caso	Caso promedio	Peor caso	Espacio
$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$

Desglose:

- Primera fase $O(n)$: recorre los n elementos para llenar el arreglo de conteo.
- Segunda fase $O(k)$: recorre las k cubetas para restaurar los elementos en orden.
- Total $O(n+k)$: si $k = O(n)$, el algoritmo es efectivamente $O(n)$ — lineal.
- Rompe la barrera teórica de $O(n \log n)$ al no basarse en comparaciones. Limitación: solo funciona con enteros en un rango conocido y consume $O(k)$ de memoria.

8. Radix Sort

Descripción

Ordena números procesando sus dígitos de menor a mayor significancia (unidades → decenas → centenas...). En cada pasada agrupa los números en 10 cubetas según el dígito analizado, usando Counting Sort como subrutina.

Pseudocódigo

```
crea 10 cubetas (colas) para cada dígito (del 0 al 9)
para cada colocación de dígito
    para cada e en la lista, mueve e a su cubo
    para cada cubo b, comenzando por el dígito más pequeño
        Mientras b no esté vacío, restaura e a la lista.
```

Análisis de complejidad — Notación Big O

Mejor caso	Caso promedio	Peor caso	Espacio
$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

Desglose:

- Se realizan k pasadas (una por dígito del valor máximo). Cada pasada cuesta $O(n+b)$ donde b es la base (10 para decimal).
- Total $O(k \times (n+b))$. Para base 10 y k constante (e.g., 6 dígitos para millones), el algoritmo es efectivamente $O(n)$.
- También supera la barrera $O(n \log n)$ al no comparar elementos directamente.
- Limitación: solo funciona con enteros y requiere memoria adicional $O(n+k)$. Para datos con pocos dígitos es muy eficiente.

Análisis Comparativo

La siguiente tabla resume la complejidad temporal y espacial de los ocho algoritmos, junto con su estabilidad. Un algoritmo es estable si preserva el orden relativo de elementos con el mismo valor.

Algoritmo	Mejor caso	Promedio	Peor caso	Espacio	Estable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Random QSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Sí
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Sí

Observaciones generales

Algoritmos simples (Bubble, Selection, Insertion Sort): los tres tienen complejidad $O(n^2)$ en promedio y peor caso, lo que los hace poco prácticos para conjuntos de datos grandes. Pero Insertion Sort es el más eficiente del grupo para listas pequeñas o casi ordenadas, con $O(n)$ en el mejor caso. Selection Sort destaca por minimizar el número de intercambios (siempre $n-1$). Bubble Sort es el menos eficiente en la práctica.

Algoritmos eficientes (Merge Sort, Quick Sort, Random Quick Sort): estos tres ofrecen $O(n \log n)$ en el caso promedio, lo que los hace viables para datos a gran escala. Merge Sort es el único que garantiza $O(n \log n)$ incluso en el peor caso, pero requiere $O(n)$ de memoria adicional. Quick Sort es más rápido en promedio por sus menores constantes y es in-place, pero tiene peor caso $O(n^2)$ si el pivote se elige mal. Random Quick Sort elimina ese riesgo aleatorizando el pivote, siendo la variante más robusta en la práctica.

Algoritmos no comparativos (Counting Sort, Radix Sort): al no basarse en comparaciones, superan la barrera teórica de $O(n \log n)$. Counting Sort es $O(n+k)$ y Radix Sort es $O(nk)$, siendo ambos prácticamente lineales cuando el rango de valores es acotado. Su limitación es que solo funcionan con enteros y requieren memoria adicional proporcional al rango de valores.